

International Institute of Information Technology, Bhubaneswar

Department of Computer Science and Engineering

Object Oriented Programming Laboratory**Lab 6 – Operator Overloading****Date:** 29.09.2026**Semester:** B.Tech 3rd Semester**Group:** B2**Instructions**

- Write all programs in C++.
- Implement the specified operators using **operator overloading**.
- Use classes and objects to represent the given entities.
- Display the output in a clear and readable format.
- Do not replace the required overloaded operator with an ordinary member function.

1. Distance Addition

Create a class Distance containing feet and inches.

Overload the + operator to add two Distance objects. If the total number of inches is 12 or more, convert the excess inches into feet.

For example:

Distance 1: 5 feet 8 inches

Distance 2: 3 feet 7 inches

Result: 9 feet 3 inches

The overloaded operator should return the resulting Distance object.

2. Complex Number Subtraction

Create a class Complex containing real and imaginary parts.

Overload the - operator to subtract two complex numbers.

For example:

C1 = 8 + 5i

C2 = 3 + 2i

C1 - C2 = 5 + 3i

Display the result in a proper complex-number format.

3. Student Marks Comparison

Create a class Student containing student name and total marks.

Overload the > operator to compare two Student objects based on their total marks.

Use the overloaded operator to determine which student has higher marks.

Condition: The overloaded operator must return a bool value.

4. Negative Value Converter

Create a class `Number` containing an integer value.

Overload the unary `-` operator so that applying it to an object creates a new object containing the negative of its value.

For example:

```
Number n1 = 25;  
Number n2 = -n1;
```

```
n1 = 25  
n2 = -25
```

The original object must remain unchanged.

5. Time Addition

Create a class `Time` containing hours and minutes.

Overload the `+` operator to add two `Time` objects.

If the total number of minutes becomes 60 or more, convert the excess minutes into hours.

For example:

```
Time 1: 4 hours 45 minutes  
Time 2: 2 hours 30 minutes  
Result: 7 hours 15 minutes
```

6. Counter Increment

Create a class `Counter` containing an integer value.

Overload the increment operator to support both prefix and postfix forms:

```
++c;  
c++;
```

Both operations should increase the counter value by 1.

Display the value before and after each operation.

***Hint:** Prefix and postfix increment operators require different function signatures.*

7. Date Equality Checker

Create a class `Date` containing day, month, and year.

Overload the `==` operator to determine whether two `Date` objects represent the same date.

For example:

```
Date 1: 15 08 2026  
Date 2: 15 08 2026
```

```
Output: Both dates are equal.
```

The overloaded operator should return a Boolean result.

8. Inventory Combination

Create a class `Item` containing item name, price, and quantity.

Overload the `+` operator to combine two `Item` objects.

If both objects represent the same item and have the same price, return a new object containing the combined quantity.

If the items are different, display an appropriate message.

Condition: The original objects must not be modified.

9. Temperature Comparison

Create a class `Temperature` containing temperature in Celsius.

Overload both the `<` and `>` operators to compare two `Temperature` objects.

Use these overloaded operators to determine whether the first temperature is lower than, higher than, or equal to the second temperature.

***Hint:** The overloaded comparison operators should return Boolean values.*

10. Shopping Cart Calculator

Create a class `Product` containing product name, price, and quantity.

Overload the `+` operator to combine two products having the same name and price by adding their quantities.

Also overload the `>` operator to compare two products based on their total value:

`Total Value = price * quantity`

Use both overloaded operators in the `main()` function to demonstrate their working.

Condition: The original product objects must remain unchanged after addition.